

Plant Reliability Agent

발전소 비계획 정지를 줄이기 위해, 현장 안에서 원인 조사를 돕는 에이전트

2026 NVIDIA Korea Agentic AI Hackathon · 공개 데이터 기반 POC

NVIDIA 스택: Nemotron 3 Ultra(NIM)가 도구를 고르고 점검안을 쓰며, NeMo Agent Toolkit이 읽기 도구를 실행하고, OpenShell로 읽기 전용 정책을 별도 시험했다([자세히](#)).

Team Sona · NVIDIA Korea Agentic AI Hackathon 2026

GitHub github.com/juyoungml/nvidia-hackathon (코드·평가·최신 문서)

Live demo juyoung.site/nvidia-hackathon

문제: 멈추면 비싸고, 판단할 자료는 흩어져 있다

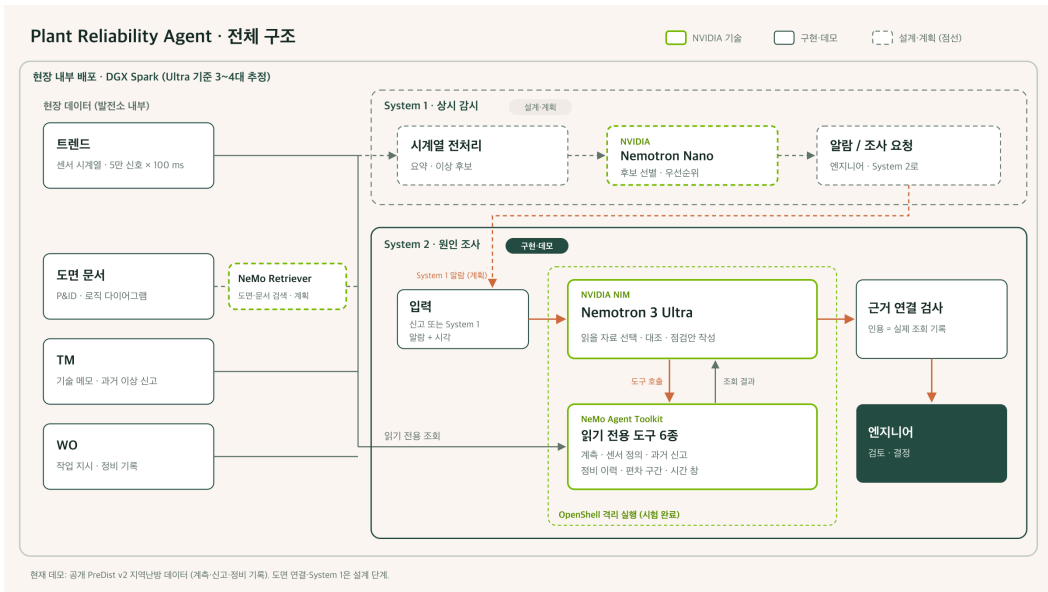
계획에 없던 설비 정지(비계획 정지)는 비용이 크다. Siemens의 *True Cost of Downtime 2024*에 따르면 포춘 글로벌 500 기업은 비계획 정지로 연 약 1.4조 달러, 매출의 약 11%를 잃는다(발전소만이 아닌 대형 산업 전반 기준, [출처](#)).

정지를 줄이려면 이상이 생겼을 때 원인을 빨리 좁혀야 한다. 그런데 현장에서는 여러 이슈가 동시에 생기고, 판단에 필요한 자료가 흩어져 있다. P&ID·로직 다이어그램 같은 도면, 기술 메모와 정비 요청(TM), 작업 지시(WO), 시간별 계측 트렌드를 사람이 하나씩 찾아 맞춰 봐야 한다. 계측값이 정상처럼 보일 때는 더 어렵다. 계기판에 없는 정보를 현장에서 확인해야 한다는 판단 자체가 필요하다.

제약: 발전소에서는 클라우드 AI를 쓰기 어렵다

발전 설비는 국가 핵심 기반시설이다. 제어·운영망은 외부망과 분리하고 출입 지점을 통제하는 것이 원칙이다. 예를 들어 북미 전력망 의무 기준 NERC CIP-005-7은 전력 계통 사이버 자산을 정의된 전자 보안 경계 안에 두고 모든 출입을 통제하도록 한다([출처](#)). 국내 국가·공공기관은 오랫동안 망분리를 적용해 왔고, 국정원 N2SF 가이드라인은 이를 자료 중요도별 차등 통제로 바꾸는 중이다([출처](#)). 어느 쪽이든 현장 자료를 외부 클라우드 AI에 그대로 보내기는 어렵다. **현장 안에 설치할 수 있는 모델이 필요하다.**

해법: 현장 설치형 Nemotron + DGX Spark, 두 부분 구조



전체 구조

	System 1 · 상시 감시	System 2 · 원인 조사
하는 일	트렌드 데이터를 늘 보고 이상을 알린다	문제가 생기면 도면·TM·WO·트렌드를 연결해 원인 파악을 돕고, 다음 점검을 근거와 함께 제안한다
요구	신호 5만 개 × 0.1초 = 초당 50만 관측값, 낮은 지연, 작은 모델(Nemotron Nano 등)	여러 자료를 깊게 읽고 추론, 근거 추적
상태	설계와 이전 소규모 실험 단계(<u>처리량 모델, 이전 Nano 실험</u>)	이번에 구현·평가한 부분 (Nemotron 3 Ultra)

왜 나뉘냐: 상시 감시는 빠르고 가벼워야 하고, 원인 조사는 깊게 읽어야 한다. 초당 50만 관측값은 큰 모델에 그대로 보낼 수 없다. 시계열 코드가 이상 후보를 추리고, 작은 모델이 우선순위를 정하고, 큰 모델은 선별된 사건만 조사한다. 처리량 모델의 예시 가정(측정 아님)에서는 모든 구간을 Ultra로 보내면 분당 6,000 요청, 전처리와 Nano 선별 후에는 분당 약 6 요청이다.

DGX Spark 몇 대가 필요한가(산술 추정, 실측 아님). DGX Spark 1대는 통합 메모리 128GB, FP4 최대 1 PFLOP이며, NVIDIA는 1대에 최대 200B, ConnectX-7로 2대를 연결하면 최대 405B 모델을 안내한다(출처). 가중치는 NVFP4 기준 파라미터당 0.5바이트, 여기에 KV 캐시·런타임 여유 20~30%를 더했다.

모델	계산	필요 대수
Nemotron 3 Ultra, 550B 전체·55B 활성 (이번에 평가한 모델, 모델 카드)	$550B \times 0.5 = 275GB \rightarrow \times 1.2-1.3 = 330-358GB \rightarrow \div 128GB = 2.6-2.8$	최소 3대, 여유 포함 약 4대
Nemotron 3 Super, 120B 전체·12B 활성 (가벼운 대안, 미평가, 모델 카드)	$120B \times 0.5 = 60GB \rightarrow \times 1.2-1.3 = 72-78GB$	1대

이번 평가는 NVIDIA NIM 호스팅 API로 실행했다. Ultra를 DGX Spark 3-4대에 나눠 올리는 구성은 NVIDIA가 안내한 2대 구성을 넘어서므로 실측으로 확인해야 한다.

데이터: 공개 데이터로 벤치마크를 만들었다

고객 발전소 데이터는 공개할 수 없다. 그래서 설비 이상 신고와 계측·정비 기록이 함께 있는 공개 지역난방 데이터 [PreDist v2](#)(CC BY 4.0)로 평가용 사건을 만들었다. 지역난방 서브스테이션은 공급온도·설정값·유량을 기록하고, 고객 신고와 정비 기록이 남는다. 공급온도가 설정값을 잘 따라가도 고객은 난방이 안 된다고 할 수 있어, “데이터에 없는 것을 현장에서 확인해야 한다”는 판단을 시험하기 좋다.

실제 사례: 신고 52

2016년 12월 12일 15:55, 한 서브스테이션에 난방 불량 신고가 들어왔다. 신고 전 24시간 동안 144개의 계측이 있고, 2차측 공급온도는 설정값과 평균 0.31°C 차이로 잘 맞았다. 계측만 보면 정상이다.

에이전트가 정리한 내용은 다음과 같다.

데이터로 확인되는 것	데이터로는 모르는 것	그래서 다음에 확인할 것
시간대별 2차측 공급온도와 설정값	실내·라디에이터의 실제 온도	영향받은 공간과 말단 온도
1차측 네트워크 유량	고객 쪽 2차 회로 유량	2차측 유량을 측정해 운전 범위와 비교
설정값이 63.0°C에서 68.6°C로 바뀜	자동 제어인지 수동 변경인지	제어기 설정·경보·변경 이력

원인을 단정하거나 설정을 바꾸라고 지시하지 않는다. 엔지니어가 검토할 점검안이다. 신고 이후의 진단이나 조치 기록은 입력에서 뺐다. 원본은 [PreDist v2](#), 재생 입력은 [replay-52.json](#)에 있다.

System 2 작동 방식

1. **신고 시점을 기준으로 자료를 준비한다.** 설비, 신고 내용, 접수 시각, 볼 수 있는 계측 구간을 정한다. 그 이후 정보는 넣지 않는다.
2. **Nemotron 3 Ultra가 필요한 자료를 고른다.** 최근 계측, 과거 신고, 정비 이력, 센서 정의, 시간대별 온도·설정값, 편차 구간을 읽는 도구 6종 중에서 선택한다. 계획 단계는 최대 6회다.
3. **점검안을 근거와 함께 낸다.** 모델은 점검 항목, 이유, 참고한 기록 ID를 출력한다. 프로그램이 그 ID를 원본 값에 연결해서, 제안과 근거가 어긋나지 않게 한다.
4. **검사 후 사람에게 넘긴다.** 인용한 기록이 실제로 조회한 것인지, 형식이 맞는지 자동으로 확인한다. 시간이나 수치 설명에 근거가 빠진 것 같으면 경고를 붙인다.

조회 한도를 다 쓰고도 에이전트가 조사를 끝내지 않으면, 그때까지 모은 자료로 정리하게 했다(`bounded_finalize_v2`). 이 경우 “스스로 끝낸 것이 아니라 한도에 도달했다”는 표시를 남긴다. 사례 52에서 이 방식으로 도구 6회 조회 후 27.8초 만에 관측 12개와 점검 3개가 나왔다([실행 기록](#)).

설비 도메인을 반영한 설계

측정 위치를 헛갈리지 않게 했다. 1차측 유량을 2차측 유량으로, 공급온도를 실내 온도로 착각하지 않도록 센서의 의미와 한계를 도구 설명에 넣었다.

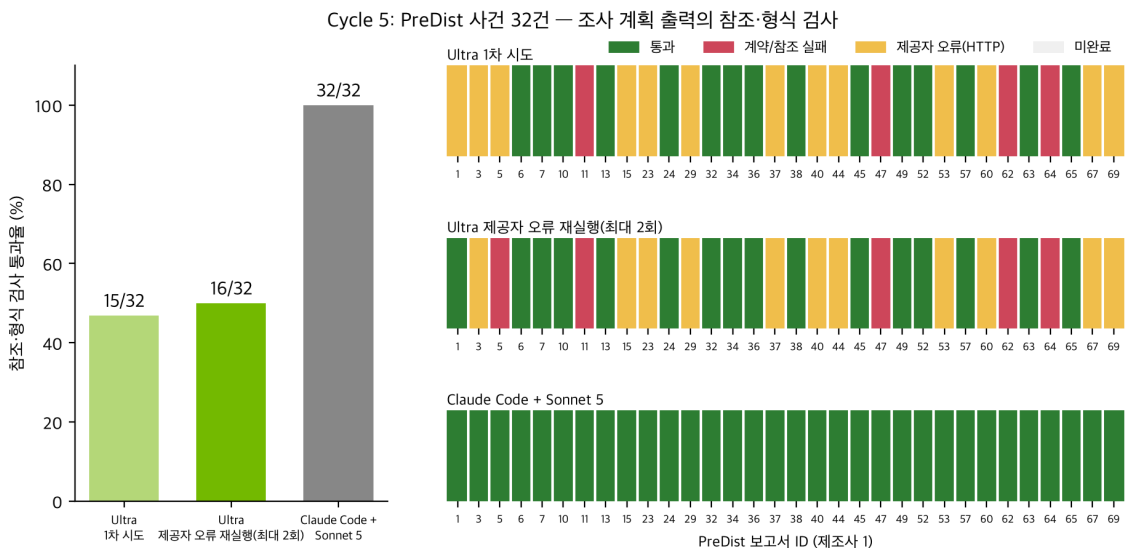
시간을 따로 다룬다. 최솟값·최댓값과 처음·마지막 값을 구분하고, 같은 시각의 공급온도와 설정값을 짝지어 조회할 수 있다. 결측이나 불규칙한 간격도 그대로 보여준다.

모든 제안에서 원본으로 돌아갈 수 있다. 기록 ID마다 출처, 필드, 시각, 값을 담았다. 화면에서 점검 항목을 누르면 그 판단에 쓴 원본 값이 바로 열린다.

평가 결과

사건 32건 평가

Sonnet 5는 비교를 위한 상한선입니다. 클라우드 최고 수준 모델이지만 보안 때문에 발전소 안에서는 쓸 수 없습니다. 그래서 질문은 “누가 이기나”가 아니라 “현장에 설치할 수 있는 모델이 이 상한선에 얼마나 가까운가”입니다. **Ultra가 끝까지 작성한 점검안 15개는 모두 근거 검사를 통과했습니다.** 조회하지 않은 기록을 지어내 인용한 경우는 0건입니다. 발전소에서 가장 위험한 것은 시가 그럴듯한 근거를 지어내는 것이기 때문에, 이 점이 현장에서 믿고 쓸 수 있는 이유입니다. **남은 격차는 판단력이 아니라 실행 안정성에서 나옵니다.** 놓친 17건 중 13건은 호스팅 API 오류로 답을 받지 못한 경우이고, 4건은 시간 구간을 한도보다 길게 요청한 한 가지 도구 실수였습니다. 현장 배포로 외부 API 요인을 없애고, 도구가 요청 범위를 자동으로 맞추도록 보완할 예정입니다(보완 후 성능은 아직 측정 전).



PreDist 사건 32건 근거 연결 검사 결과

	통과	통과율	API 연결 오류	오류 제외 통과율	조사 시간 중앙값
Nemotron 3 Ultra	15/32	47%	13	15/19 (79%)	45.2초
Claude Code + Sonnet 5	32/32	100%	0	32/32 (100%)	48.2초

공개 지역난방 사건 32건으로 평가를 넣었습니다(사건당 1회 실행). 같은 조건에서 Claude Sonnet 5는 32건 모두 근거 연결 검사를 통과했고, Nemotron 3 Ultra는 15건(47%)을 통과했습니다. Ultra가 놓친 17건 중 13건은 답이 틀린 것이 아니라 호스팅 API 연결 오류(HTTP)로 결과를 받지 못한 경우입니다. 이 오류를 빼면 Ultra는 19건 중 15건(79%)을 통과했고, 결과를 받은 실행의 조사 시간 중앙값은 Ultra 45초, Sonnet 48초였습니다. 현장에 직접 배포하면 외부 API 연결 오류

요인은 줄일 수 있고, 1차 시도에서 남은 4건(재실행 후 5건)의 형식·근거 실패는 오프라인 환경에서 보완할 예정입니다. 이 검사는 진단 정확도가 아니라, 제안이 실제 기록을 제대로 가리키는지를 본 것입니다. API 오류가 난 사례를 재실행하면 Ultra는 16/32였고, 결과를 받은 21건 중 16건(76%)이 통과했습니다. 실제 모델 실패 5건은 모두 시간 구간 조회 한도(24행)를 넘겨 요청한 경우였습니다. 사건당 1회 실행이며, 두 경로는 모델과 실행 환경이 함께 다릅니다. 자세한 내용은 [평가 결과](#)와 [사전 프로토콜](#)에 있습니다.

사건 4건 비교

같은 사건, 같은 자료, 같은 출력 형식으로 Ultra 경로와 Claude Code + Sonnet 5 경로를 비교했다. 사례와 데이터 해시는 실험 전에 고정했다([프로토콜](#)). Sonnet 쪽은 파일 읽기·검색과 구조화 출력만 쓰게 했다. 모델과 실행 환경이 함께 달라서 어느 쪽 효과인지는 분리되지 않는다.

근거 연결 검사 통과	Ultra	Sonnet 5
개발에 쓴 사건 (52, 3)	1/2	2/2
처음 보는 설비 사건 (29, 47)	1/2	2/2
합계	2/4	4/4

Ultra가 놓친 2건은 모두 조회 6회를 다 쓰고 나서 조사 종료를 선언하지 못한 경우였다. 앞서 말한 한도 도달 후 정리 방식이 이 문제를 고친 것이다. 수정 결과는 위 표에 넣지 않았다.

둘 다 결과를 낸 2건은 이름을 가리고 별도 AI 검토자에게 점검 내용을 평가받았다. 점수는 Ultra 41/48, Sonnet 42/48로 비슷했고, 양쪽 모두 점검 6개 중 3개에서 중요한 근거가 빠져 있었다([평가 결과](#)). 표본이 작아서 우열을 말하기는 어렵다.

부족한 부분은 오프라인 환경에서 보완할 예정이다.

실험 중 발견한 문제도 기록해 두었다. 첫 평가 자료에서 점검 본문이 빠진 것을 발견해 다시 평가했고, 요청 로그에 이후 메시지가 섞이는 버그를 고쳤다. 자세한 내용은 [전체 결과](#)에 있다.

사용한 NVIDIA 기술

모델(Nemotron 3 Ultra·NIM), 도구 실행(NeMo Agent Toolkit), 안전한 실행 환경(OpenShell)까지 에이전트 실행 생태계가 갖춰져 있어, 로컬·보안 환경에 맞는 에이전트를 짧은 시간에 구성할 수 있었다. 이 프로젝트는 범용 챗봇이 아니다. NVIDIA 모델이 실제 도구를 직접 고르고, 그 도구가 돌려준 기록을 근거로 점검안을 쓰는 에이전트 구조다.

기술	이 프로젝트에서 맡은 일	상태	확인할 곳
Nemotron 3 Ultra (NVIDIA NIM 호스팅 API, <code>nvidia/nemotron-3-ultra-550b-a55b</code>)	에이전트의 두뇌. 읽기 전용 도구 6종 중 무엇을 부를지 고르고(tool calling), 최대 6번의 계획 요청으로 결과를 비교한 뒤, 최종 점검안을 JSON 스키마 구조화 출력으로 쓴다. 모든 요청·도구 결과·근거 검사를 기록한다.	모든 데모 실행의 핵심 경로	실행 코드 , 사례 52 기록 , 구조

기술	이 프로젝트에서 맡은 일	상태	확인할 곳
NeMo Agent Toolkit (NAT) 1.8.0	읽기 도구 6종을 등록하고 실행한다. 키 없이 도는 동등성 시험에서 직접 호출과 결과가 같았고, 별도 사례 52 개발 실행에서 모델이 고른 6번의 조회를 NAT가 실행했으며 점검안이 근거 검사를 통과했다. 계획 루프 자체는 우리 Python 코드다.	연동 완료 (선택 backend <code>--read-backend nat</code>)	NAT 연동, 동등성 기록, 사례 52 기록
OpenShell 0.1.2	공개 시험 자료로 정책을 따로 시험했다. 허용된 읽기는 통과하고, 비허용 읽기·쓰기·직접 TCP 연결은 막혔다. 전체 추론 경로에는 아직 적용하지 않았다.	별도 시험 완료	시험 설명, 정책, 기록
Nemotron Nano, NeMo Retriever	Nano는 System 1로서 많은 신호를 상시 감시하고 관련 자료를 재정렬하는 역할, Retriever는 문서·도면 검색 역할로 계획했다. 이전 Nano 분류 실험 기록은 남아 있지만 주 경로에는 없다.	계획	구조, 이전 Nano 실험

API 키와 회사·고객 자료는 제출물에 없다. PreDist는 CC BY 4.0 공개 데이터다. DOE 증기 계통도는 공개 도면 탐색 예시로만 넣었고 PreDist 설비와는 관계가 없다.

다음 단계

목표는 엔지니어가 다음 점검을 정할 때 자료를 찾고 대조하는 시간을 줄이는 것이다. 모델 응답 시간은 이 시간을 대신하지 못하므로 따로 측정해야 한다.

다음 실험에서는 같은 신규 사건을 현장 경험자에게 주고, 기존 방식과 Plant를 번갈아 쓰게 한다. 첫 번째 쓸 만한 점검안이 나오기까지의 시간, 빠진 근거, 불필요한 점검, 근거 없는 확신을 함께 본다. 평가자는 어느 방법인지 모르게 한다.

그다음은 발전소 설비로의 확장이다. DGX Spark 위에 모델을 올려 현장 내부에서 실측하고, System 1(Nemotron Nano)이 신호 5만 개를 상시 감시하다가 이상 징후가 보이면 System 2 조사를 시작하는 구조를 만들 계획이다.

Plant는 설비를 자동으로 제어하지 않고, 현장 운전 절차를 대신하지 않는다. 판단은 엔지니어가 한다.